GRAPHUG: ENTORNO MATEMÁTICO INTERACTIVO Y CALCULADORA GRÁFICA

Andrés Torres Ceja

Lic. Ingeniería en Sistemas Computacionales
Ciencias de la Computación "Soft Computing CI"
Universidad de Guanajuato - DICIS

Salamanca, Guanajuato, México | A domingo 26 de abril de 2026 | a.torresceja@ugto.mx

Resumen (Abstract) — El presente documento describe el diseño, desarrollo y fundamentos matemáticos de **GraphUG**, una aplicación de escritorio de código abierto que funge como entorno matemático interactivo y calculadora gráfica, concebida como una alternativa moderna y simplificada a MATLAB. GraphUG integra un analizador sintáctico LALR(1) construido con Lark, un evaluador de expresiones basado en NumPy, un motor de álgebra simbólica sustentado en SymPy, y un sistema de renderizado dual 2-D/3-D implementado con PyQtGraph y OpenGL. La arquitectura del sistema sigue estrictamente el patrón Modelo-Vista-Controlador (MVC) con inyección de dependencias explícita, garantizando separación de responsabilidades y extensibilidad. El proyecto abarca más de 130 funciones numéricas integradas, soporte para cálculo vectorial simbólico, transformadas de Laplace, y más de 20 primitivas de graficación que incluyen superficies 3-D, curvas paramétricas, campos vectoriales y mapas de calor. Se documentan exhaustivamente los fundamentos de geometría analítica, análisis numérico y teoría de lenguajes formales que sustentan cada componente del sistema.

Palabras clave: Entorno matemático interactivo, LALR(1), graficación computacional, geometría analítica, álgebra simbólica, Python, NumPy, SymPy, PyQtGraph, MVC.

I. INTRODUCCIÓN

A. Contexto y Motivación

Los entornos matemáticos interactivos como MATLAB, Mathematica y Octave constituyen herramientas fundamentales en ciencias de la computación, ingeniería y matemáticas aplicadas. Sin embargo, estas herramientas suelen ser propietarias, costosas o de difícil extensión. GraphUG surge como una alternativa de código abierto que combina evaluación numérica de alto rendimiento, álgebra simbólica y graficación avanzada en una interfaz moderna y accesible.

El objetivo principal de GraphUG es proporcionar un entorno donde el usuario pueda:

1. Evaluar expresiones matemáticas arbitrarias con aritmética de punto flotante IEEE 754.
2. Manipular vectores y matrices mediante operaciones de álgebra lineal.
3. Realizar cálculo diferencial e integral tanto numérico como simbólico.
4. Graficar funciones en 2-D y 3-D con primitivas variadas.
5. Ejecutar operaciones de cálculo vectorial (gradiente, divergencia, rotacional, laplaciano).

B. Contribuciones del Proyecto

Las contribuciones principales de este trabajo son:

- **Diseño de un lenguaje matemático completo** con gramática LALR(1) que soporta aritmética, comparaciones, operadores lógicos, asignación de variables, vectores, matrices, funciones, operador ternario y operador pipe.
- **Arquitectura MVC con inyección de dependencias** que desacopla completamente la capa de presentación del motor matemático.
- **Motor de evaluación híbrido** que combina computación numérica (NumPy) con álgebra simbólica (SymPy).
- **Sistema de renderizado dual** con soporte para más de 20 primitivas de graficación en 2-D y 3-D.
- **Aceleración GPU transparente** mediante CuPy para operaciones de álgebra lineal en arreglos grandes.

C. Estructura del Documento

La Parte I cubre la arquitectura del software, la gramática formal del lenguaje, el proceso de análisis sintáctico y el motor de evaluación. La Parte II aborda los fundamentos matemáticos de la graficación, el álgebra simbólica, el cálculo vectorial y el sistema de renderizado.

II. MATERIALES Y MÉTODOS

A. Pila Tecnológica y Dependencias

El sistema se implementa en **Python 3.12+** y emplea las siguientes dependencias:

Dependencia	Versión	Rol en el Sistema
PySide6 $\geq$ 6,6,0	Framework Qt6	Interfaz gráfica de usuario (GUI)
PyQtGraph $\geq$ 0,13,3	Graficación 2-D/3-D	Renderizado de alto rendimiento
PyOpenGL $\geq$ 3,1,6	Bindings OpenGL	Soporte 3-D (superficies, wireframes)

Dependencia	Versión	Rol en el Sistema
NumPy $\geq$ 1,26,0	Computación numérica	Operaciones vectorizadas y álgebra lineal
SymPy $\geq$ 1,12	Álgebra simbólica	Diferenciación, integración, simplificación
Lark $\geq$ 1,1,9	Generador de parsers	Análisis sintáctico LALR(1)
pytest $\geq$ 7,4	Framework de pruebas	126+ pruebas unitarias

B. Arquitectura del Software

1) Patrón Modelo-Vista-Controlador (MVC)

GraphUG implementa una arquitectura **MVC estricta con Clean Architecture** y principios SOLID. La separación se logra mediante:

- **Capa de Dominio** (`app/core/`): Define interfaces abstractas (`IEvaluator`, `IRenderer`, `IController`), modelos de datos (`MathResult`, `PlotCommand`, `Expression`) y excepciones personalizadas. Esta capa **no importa ninguna dependencia externa**.
- **Capa de Infraestructura** (`app/parser/`, `app/renderer/`, `app/math_engine/`): Implementaciones concretas de las interfaces del dominio.
- **Capa de Presentación** (`app/gui/`): Widgets PySide6 que constituyen la vista.
- **Capa de Aplicación** (`app/controllers/`): El `MainController` que orquesta el flujo.

El flujo de datos se describe formalmente como:

$$\text{EditorPanel} \xrightarrow{\text{input_submitted(str)}} \text{MainController} \xrightarrow{\texttt{evaluate()}} \text{MathEvaluator} \xrightarrow{\text{MathResult}} \text{Renderer} \xrightarrow{\texttt{render()}} \text{Canvas}$$

2) Inyección de Dependencias

El punto de entrada (`main.py`) realiza el cableado explícito del grafo de objetos:

```
evaluator = MathEvaluator()                          # IEvaluator
renderer = PyQtGraphRenderer(canvas.plot_ widget)      # IRenderer (2-D)
renderer_ 3d = PyQtGraph3DRenderer(canvas.gl_ widget)   # IRenderer (3-D)
controller = MainController(evaluator, renderer, renderer_ 3d=renderer_ 3d)
```

No se utilizan singletons, localizadores de servicios ni estado global. Cada objeto recibe sus dependencias a través de su constructor.

3) Interfaces Abstractas

La interfaz `IEvaluator` define el contrato:

$$\texttt{evaluate}(s : \text{str}) \to \text{MathResult}$$

donde `MathResult` es un Data Transfer Object (DTO) con la estructura:

$$\text{MathResult} = \langle \text{value}, \text{plot_commands}, \text{output_text}, \text{error} \rangle$$

La invariante es: si error $\neq$ None, los demás campos no contienen datos significativos.
La interfaz `IRenderer` expone:

$$\texttt{render}(c : \text{PlotCommand}) \rightarrow \text{None}$$

donde `PlotCommand` encapsula el tipo de primitiva (`PlotKind`), los datos numéricos y metadatos de estilo:

$$\text{PlotCommand} = \langle \text{kind} : \text{PlotKind}, \text{data} : \text{dict}, \text{label} : \text{str}, \text{color} : \text{str}, \text{line_width} : \text{float} \rangle$$

C. Gramática Formal del Lenguaje

1) Definición EBNF

El lenguaje GraphUG se define mediante una gramática libre de contexto procesada por un parser LALR(1). La jerarquía de precedencia de operadores (de menor a mayor) es:

$$\text{ternario} < \text{pipe} < \text{or} < \text{and} < \text{not} < \text{comparación} < \text{suma} < \text{producto} < \text{potencia} < \text{unario} < \text{postfijo} < \text{átomo}$$

La gramática formal en notación EBNF se define como:

```
\small
start       → statement
statement   → assignment | expr
assignment  → NAME '=' expr

expr        → ternary_expr
ternary_expr → pipe_expr '?' pipe_expr ':' ternary_expr | pipe_expr
pipe_expr   → or_expr | pipe_expr '|>' NAME
or_expr     → and_expr | or_expr 'or' and_expr
and_expr    → not_expr | and_expr 'and' not_expr
not_expr    → comparison | 'not' not_expr
comparison  → sum | comparison ('==' | '!=' | '<' | '>' | '<=' | '>=') sum
sum         → product | sum ('+' | '-') product
product     → power | product ('*' | '/' | '%') power
power       → unary | unary '^' power    (asociatividad derecha)
unary       → '-' atom | '+' atom | atom
atom        → atom_base | atom_base '[' expr ']' | atom_base '(' arglist ')'
atom_base   → NUMBER | STRING | NAME '(' arglist ')' | NAME | '[' bracket_body ']' | '(' expr '
bracket_body → row ';' row (';' row)*  |  arglist
```

2) Análisis LALR(1) con Lark

El parser LALR(1) (*Look-Ahead Left-to-right Rightmost derivation*) procesa la gramática en tiempo lineal $O(n)$ con respecto a la longitud de la entrada. La clase de gramáticas LALR(1) es un subconjunto de las gramáticas LR(1) que utiliza tablas de análisis más compactas.

Formalmente, un parser LALR(1) opera sobre una tabla de acciones ACTION[s,a] y una tabla de transiciones GOTO[s,A] donde:

- s es el estado actual de la pila.
- a es el token de entrada (*lookahead*).
- A es un símbolo no terminal.

La decisión entre *shift* (desplazar) y *reduce* (reducir) se toma en $O(1)$ por token.

Lark implementa una optimización clave: el `Transformer` se ejecuta **inline** durante el análisis sintáctico (parámetro `transformer=`), convirtiendo cada nodo del árbol de derivación directamente en un nodo AST tipado, eliminando la necesidad de una segunda pasada:

$$\text{Texto} \xrightarrow[\text{LALR(1)}]{\text{Lark}} \text{Parse Tree} \xrightarrow[\text{inline}]{\text{MathTransformer}} \text{AST tipado}$$

3) Árbol de Sintaxis Abstracta (AST)

Los nodos del AST se definen como `dataclass` inmutables con metadatos de ubicación:

Nodo	Campos	Descripción
`NumberNode`	`value: float`	Literal numérico
`StringNode`	`value: str`	Literal de cadena
`SymbolNode`	`name: str`	Referencia a variable/función
`BinaryOpNode`	`op, left, right`	Operación binaria infija
`UnaryOpNode`	`op, operand`	Operación unaria prefija
`FuncCallNode`	`name, args[]`	Invocación de función
`VectorNode`	`elements[]`	Literal vectorial $[e_1, e_2, \ldots]$
`MatrixNode`	`rows[][]`	Literal matricial $[r_1; r_2; \ldots]$
`AssignmentNode`	`name, value`	Asignación name = expr
`IndexAccessNode`	`obj, index`	Acceso por índice $v[i]$
`TernaryNode`	`condition, if\_true, if\_false`	Condicional ternario
`PipeNode`	`value, func\_name`	Operador pipe expr $\vert>$ func

D. Motor de Evaluación

1) *Despacho por Pattern Matching Estructural*

El evaluador utiliza *structural pattern matching* de Python 3.10 para despachar la evaluación según el tipo concreto de nodo AST:

$$\texttt{_eval_node}(n) = \begin{cases} n.\text{value} & \text{si } n \in \text{NumberNode} \\ \text{scope}[n.\text{name}] & \text{si } n \in \text{SymbolNode} \\ \text{eval}(n.\text{left}) \oplus \text{eval}(n.\text{right}) & \text{si } n \in \text{BinaryOpNode} \\ f(\text{eval}(a_1), \ldots, \text{eval}(a_k)) & \text{si } n \in \text{FuncCallNode} \end{cases}$$

donde $\oplus$ representa el operador binario correspondiente.

2) *Ámbito de Variables y Estado de Sesión*

El evaluador mantiene un diccionario de ámbito `_scope` que persiste a lo largo de una sesión:

$$\text{scope} = \text{BUILTINS} \cup \text{USER_VARS}$$

Los nombres integrados (*built-ins*) están protegidos contra reasignación. El método `reset_state()` restaura el ámbito a su estado inicial.

3) *Funciones Integradas*

GraphUG provee **más de 130 funciones integradas** organizadas en categorías:

Trigonométricas: Las funciones trigonométricas fundamentales se implementan como wrappers sobre NumPy, operando element-wise sobre arreglos:

$$\sin, \cos, \tan, \arcsin, \arccos, \arctan, \text{atan2}, \sinh, \cosh, \tanh, \sec, \csc, \cot, \text{sinc}$$

donde $\sec(x) = \frac{1}{\cos(x)}$, $\csc(x) = \frac{1}{\sin(x)}$, $\cot(x) = \frac{\cos(x)}{\sin(x)}$.

Constructores de Arreglos: La función `linspace` genera n puntos equiespaciados en $[a,b]$:

$$x_k = a + k \cdot \frac{b-a}{n-1}, \quad k = 0, 1, \ldots, n-1$$

Álgebra Lineal: Determinante, inversa, eigenvalores, SVD, factorización QR, Cholesky, con aceleración GPU transparente para arreglos grandes (≥ 5000 elementos).

Estadística: Media, varianza, desviación estándar, mediana, percentiles, covarianza, correlación.

4) *Procesamiento Multi-sentencia*

El evaluador procesa múltiples sentencias separadas por punto y coma, respetando la sintaxis matricial donde el punto y coma separa filas:

```
\small
_split_statements("a = 1; b = [1,2; 3,4]; a + det(b)")
→ ["a = 1", "b = [1,2; 3,4]", "a + det(b)"]
```

El algoritmo mantiene un contador de profundidad de corchetes para distinguir separadores de sentencias de separadores de filas matriciales.

E. Jerarquía de Excepciones

GraphUG define una jerarquía de excepciones con sugerencias inteligentes:

```
\small
GraphUGError (base)
├── ParseError           → Errores de sintaxis con ubicación (línea, columna)
├── EvaluationError      → Errores de evaluación (tipos, operaciones)
│   ├── UndefinedSymbolError → Con sugerencia "Did you mean...?" (difflib)
│   └── DimensionError   → Incompatibilidad de dimensiones
```

La clase `UndefinedSymbolError` utiliza `difflib.get_close_matches()` con umbral de similitud 0.6 para sugerir correcciones.

F. Sistema de Pruebas

El proyecto incluye **126+ pruebas unitarias** organizadas en 6 fases que cubren:

- Aritmética básica, variables, funciones integradas
- Vectores, matrices, comparaciones, módulo
- Cadenas, punto y coma, álgebra simbólica
- Graficación avanzada (fplot, polar, parametric, surface)
- Funciones extendidas (señales, conjuntos, complejos)
- Cálculo vectorial simbólico (gradiente, divergencia, rotacional)

Las pruebas se ejecutan con `pytest` con cobertura de código (`pytest-cov`).

III. RESULTADOS — ARQUITECTURA DE COMPONENTES

A. Estructura del Proyecto

```
GraphUG/
├── main.py                    # Punto de entrada — cableado DI
├── app/
│   ├── core/                  # Capa de dominio (sin imports externos)
│   │   ├── interfaces/        #   ABC: IEvaluator, IRenderer, IController
│   │   ├── models/            #   DTOs: MathResult, PlotCommand, Expression
│   │   └── exceptions/        #   Jerarquía de errores personalizados
│   ├── parser/                # Infraestructura: Lark + MathEvaluator
│   │   ├── grammar/           #   math_ grammar.lark (LALR)
│   │   ├── ast_ nodes.py       #   Dataclasses AST tipados
│   │   └── evaluator.py       #   Implementación concreta de IEvaluator
│   ├── renderer/              # Infraestructura: PyQtGraph
│   │   ├── pyqtgraph_ renderer.py      # Renderizador 2-D
│   │   └── pyqtgraph_ 3d_ renderer.py   # Renderizador 3-D (OpenGL)
│   ├── math_ engine/           # Helpers numéricos y simbólicos
│   │   ├── numerical.py       #   Wrappers NumPy
```

```
│   │   ├── symbolic.py        #   Integración SymPy
│   │   └── gpu_ backend.py     #   Aceleración GPU (CuPy)
│   ├── gui/                   # Capa de presentación: PySide6
│   │   ├── main_ window.py     #   Shell de la aplicación
│   │   ├── widgets/           #   EditorPanel, CanvasPanel, OutputPanel
│   │   ├── dialogs/           #   InsertVector, InsertMatrix, About, Settings
│   │   └── styles/            #   ThemeManager, paletas Catppuccin
│   ├── controllers/           # Capa de aplicación: MainController
│   └── utils/                 # Logger centralizado (rotating file + stderr)
├── tests/                     # 126+ pruebas unitarias (pytest)
├── pyproject.toml             # Configuración del proyecto (PEP 621)
└── requirements.txt           # Dependencias de ejecución
```

B. Señales y Slots — Topología de Comunicación

El sistema utiliza el mecanismo de **señales y slots** de Qt para comunicación desacoplada:

Señal	Emisor	Receptor	Datos
`input_submitted`	`EditorPanel`	`MainController.handle_input`	`str`
`result_ready`	`MainController`	`MainWindow.show_result`	`str`
`error_occurred`	`MainController`	`MainWindow.show_error`	`str`
`session_reset_req`	`MainWindow`	`MainController.reset_session`	NULL
`canvas_clear_req`	`MainWindow`	`MainController.clear_canvas`	NULL
`mode_changed`	`CanvasPanel`	NULL	`str` (“2d”/“3d”)

C. Motor de Álgebra Simbólica

El módulo `symbolic.py` actúa como fachada sobre SymPy, implementando carga perezosa (*lazy loading*) para evitar penalizar el tiempo de inicio:

Operaciones simbólicas disponibles:

Función	Operación Matemática
`simplify(.expr")`	Simplificación algebraica
`factor(.expr")`	Factorización: $x^2 - 1 \to (x-1)(x+1)$
`expand(.expr")`	Expansión: $(x+1)^2 \to x^2 + 2x + 1$
`diff(.expr", "var")`	$\frac{d}{dx} f(x)$
`integrate(.expr", "var")`	$\int f(x)\,dx$
`solve(.expr", "var")`	Resolución de ecuaciones

Función	Operación Matemática
`limit(.expr", "var", val)`	$\lim_{x\to a} f(x)$
`series(.expr", "var", pt, n)`	Serie de Taylor/Maclaurin
`gradient(.expr", "x,y,...")`	$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \ldots\right]$
`divergence("F1,F2,...", "x,y,...")`	$\nabla \cdot \mathbf{F} = \sum_i \frac{\partial F_i}{\partial x_i}$
`curl("F1,F2,F3", "x,y,z")`	$\nabla \times \mathbf{F}$
`laplacian(.expr", "x,y,...")`	$\nabla^2 f = \sum_i \frac{\partial^2 f}{\partial x_i^2}$
`laplace(.expr", "t", "s")`	$\mathscr{L}\{f(t)\}(s)$
`invlaplace(.expr", "s", "t")`	$\mathscr{L}^{-1}\{F(s)\}(t)$

D. Aceleración GPU

El módulo `gpu_backend.py` implementa aceleración transparente mediante CuPy:

$$\text{backend}(A) = \begin{cases} \text{CuPy} & \text{si GPU disponible} \wedge |A| \geq 5000 \\ \text{NumPy} & \text{en caso contrario} \end{cases}$$

Las operaciones aceleradas incluyen: FFT, multiplicación matricial, determinante, inversa, eigenvalores, SVD y resolución de sistemas lineales.

IV. RESULTADOS — FUNDAMENTOS MATEMÁTICOS DE LA GRAFICACIÓN

A. Graficación 2-D: Primitivas y Geometría Analítica

1) Graficación de Líneas (`plot`)Graficación de Líneas (plot)

La primitiva fundamental es la **curva de línea 2-D**. Dada una función $f : \mathbb{R} \to \mathbb{R}$, la graficación consiste en:

1. **Discretización del dominio:** Generar n puntos equiespaciados $\{x_k\}_{k=0}^{n-1}$ en el intervalo $[a,b]$:

$$x_k = a + k \cdot h, \quad h = \frac{b-a}{n-1}$$

2. **Evaluación vectorizada:** Calcular $y_k = f(x_k)$ para todo k simultáneamente mediante operaciones NumPy element-wise.

3. **Interpolación lineal en renderizado:** PyQtGraph conecta los puntos (x_k, y_k) mediante segmentos de línea. Para n suficientemente grande (típicamente $n = 500$ para `fplot`), la curva resultante es visualmente suave.

La función `plot(x, y)` acepta arreglos NumPy y genera un `PlotCommand` con `kind=PlotKind.LINE_2D` y `data={"x": x, ɾ": y}`.

2) Graficación de Dispersión (`scatter`)Graficación de Dispersión (scatter)

El gráfico de dispersión representa un conjunto de puntos discretos $\{(x_i, y_i)\}_{i=1}^{n}$ sin interpolación. Se implementa mediante `pg.ScatterPlotItem` con marcadores circulares de 8 píxeles.

3) Graficación de Vectores (`vector`)Graficación de Vectores (vector)

Un vector 2-D se representa geométricamente como una flecha desde un punto de origen (x_0, y_0) hasta $(x_0 + d_x, y_0 + d_y)$. El renderizado se compone de:

- **Eje (shaft):** Segmento de línea de (x_0, y_0) a $(x_0 + d_x, y_0 + d_y)$.
- **Punta de flecha:** `pg.ArrowItem` posicionada en el extremo, con ángulo calculado mediante:

$$\theta = \arctan 2(-d_y, -d_x) \cdot \frac{180}{\pi}$$

La función $\arctan 2(y, x)$ proporciona el ángulo en el rango $(-\pi, \pi]$ respetando el cuadrante correcto, a diferencia de $\arctan(y/x)$ que es ambigua.

4) Diagramas de Barras (`bar`)Diagramas de Barras (bar)

Cada barra se posiciona en x_i con altura h_i y ancho w (por defecto $w = 0{,}8$). Se utiliza `pg.BarGraphItem` que renderiza rectángulos:

$$\text{Barra}_i = \{(x, y) : x_i - w/2 \leq x \leq x_i + w/2,\ 0 \leq y \leq h_i\}$$

5) Histogramas (`hist`)Histogramas (hist)

El histograma divide los datos en B bins (por defecto $B = 20$) y cuenta las frecuencias. Dado un conjunto de datos $\{d_j\}_{j=1}^{N}$:

1. Se calculan los bordes de bins: $e_0, e_1, \ldots, e_B$ equiespaciados en $[\text{mín}(d), \text{máx}(d)]$.
2. Se cuentan las ocurrencias: $c_i = |\{j : e_{i-1} \leq d_j < e_i\}|$.
3. Los centros de bins se calculan como: $\bar{e}_i = \frac{e_{i-1}+e_i}{2}$.
4. Se renderiza como un diagrama de barras con ancho $w = 0{,}9 \cdot (e_1 - e_0)$.

6) Relleno entre Curvas (`fill_between`)Relleno entre Curvas (fill_between)

Utilizado para visualizar integrales definidas. Dadas dos curvas $y_1(x)$ e $y_2(x)$, se sombrea la región:

$$\mathscr{R} = \{(x, y) : a \leq x \leq b,\ \text{mín}(y_1(x), y_2(x)) \leq y \leq \text{máx}(y_1(x), y_2(x))\}$$

Se implementa mediante `pg.FillBetweenItem` con transparencia alpha de 0,25 (sufijo hexadecimal `40`).

7) Curvas Implícitas (`implicit`)Curvas Implícitas (implicit)

Una curva implícita se define como el conjunto de nivel cero de una función de dos variables:

$$\mathscr{C} = \{(x,y) \in \mathbb{R}^2 : f(x,y) = 0\}$$

El algoritmo de renderizado es:

1. Discretizar una rejilla de 200×200 puntos en $[x_0, x_1] \times [y_0, y_1]$.
2. Evaluar $Z_{ij} = f(X_{ij}, Y_{ij})$ en cada punto de la rejilla.
3. Utilizar `pg.IsocurveItem` con `level=0.0` para trazar la curva de nivel.

La transformación del espacio de datos al espacio de píxeles se realiza mediante:

$$s_x = \frac{x_1 - x_0}{n_{\text{cols}}}, \quad s_y = \frac{y_1 - y_0}{n_{\text{rows}}}$$

aplicada como una transformación afín `QTransform().scale(sx, sy)`.

8) Curvas de Contorno (`contour`)Curvas de Contorno (contour)

Las curvas de contorno generalizan las curvas implícitas a múltiples niveles:

$$\mathscr{C}_k = \{(x,y) : f(x,y) = \ell_k\}, \quad k = 1, \ldots, K$$

donde los niveles ℓ_k se distribuyen uniformemente en $[\text{mín}(Z), \text{máx}(Z)]$ excluyendo los extremos:

$$\ell_k = Z_{\text{mín}} + k \cdot \frac{Z_{\text{máx}} - Z_{\text{mín}}}{K+1}, \quad k = 1, \ldots, K$$

9) Campos de Pendientes (`slopefield`)Campos de Pendientes (slopefield)

Un campo de pendientes visualiza la ecuación diferencial ordinaria $\frac{dy}{dx} = f(x,y)$. En cada punto (x_i, y_j) de una rejilla de 20×20 se dibuja un segmento de línea con pendiente $f(x_i, y_j)$.

El proceso de normalización es:

1. Calcular $DY_{ij} = f(X_{ij}, Y_{ij})$ y $DX_{ij} = 1$.
2. Calcular la magnitud: $M_{ij} = \sqrt{DX_{ij}^2 + DY_{ij}^2}$.
3. Normalizar y escalar: $\text{scale} = \frac{\text{mín}(\Delta x, \Delta y)}{n_{\text{grid}} \cdot 2{,}5}$.
4. El segmento en (x_i, y_j) va de $(x_i - \tilde{dx}, y_j - \tilde{dy})$ a $(x_i + \tilde{dx}, y_j + \tilde{dy})$ donde $\tilde{dx} = \frac{DX_{ij}}{M_{ij}} \cdot \text{scale}$.

10) Campos Vectoriales 2-D (`vectorfield`)Campos Vectoriales 2-D (vectorfield)

Un campo vectorial $\mathbf{F}(x,y) = (u(x,y), v(x,y))$ se visualiza como una colección de flechas. El proceso es análogo al campo de pendientes, pero con componentes independientes u y v:

$$\text{Magnitud: } M_{ij} = \sqrt{u_{ij}^2 + v_{ij}^2}$$

Las flechas se normalizan para mantener longitud visual uniforme, preservando la dirección:

$$\hat{u}_{ij} = \frac{u_{ij}}{M_{ij}} \cdot \text{scale}, \quad \hat{v}_{ij} = \frac{v_{ij}}{M_{ij}} \cdot \text{scale}$$

11) Mapas de Calor (`heatmap`)Mapas de Calor (heatmap)

Un mapa de calor visualiza una función escalar $f(x,y)$ mediante una imagen coloreada. Se utiliza `pg.ImageItem` con la tabla de colores *viridis* de Matplotlib (256 niveles):

$$\text{color}(x,y) = \text{viridis}\left(\frac{f(x,y) - f_{\text{mín}}}{f_{\text{máx}} - f_{\text{mín}}}\right)$$

B. Coordenadas Polares (`polar`)Coordenadas Polares (polar)

La graficación polar transforma una función $r(\theta)$ en coordenadas cartesianas:

$$x(\theta) = r(\theta)\cos(\theta), \quad y(\theta) = r(\theta)\sin(\theta)$$

donde $\theta \in [\theta_0, \theta_1]$ (por defecto $[0, 2\pi]$). La curva polar se renderiza como una línea 2-D estándar en el plano cartesiano.

C. Curvas Paramétricas 2-D (`parametric`)Curvas Paramétricas 2-D (parametric)

Una curva paramétrica $\gamma(t) = (x(t), y(t))$ para $t \in [t_0, t_1]$ se discretiza en $n = 500$ puntos:

$$\gamma_k = (x(t_k), y(t_k)), \quad t_k = t_0 + k \cdot \frac{t_1 - t_0}{n-1}$$

D. Graficación Simbólica

1) Graficación de Funciones Simbólicas (`fplot`)Graficación de Funciones Simbólicas (fplot)

La función `fplot(.^expr")` utiliza el motor simbólico para convertir una expresión en cadena a una función NumPy evaluable mediante *lambdificación*:

$$\text{str} \xrightarrow{\texttt{sympify}} \text{Expr (SymPy)} \xrightarrow{\texttt{lambdify}} f : \mathbb{R}^n \to \mathbb{R}^n \text{ (NumPy)}$$

La función `lambdify` genera código Python que invoca funciones NumPy equivalentes, resultando en evaluación vectorizada de alto rendimiento.

Para garantizar que la salida sea siempre un arreglo (incluso para constantes), se aplica un wrapper de seguridad:

```
def _safe(v):
    result = fn(v)
    return np.full_like(v, result) if np.ndim(result) == 0 else np.asarray(result)
```

2) Derivada Gráfica (`plotderiv`)Derivada Gráfica (plotderiv)

`plotderiv(.^expr")` grafica simultáneamente $f(x)$ y $f'(x)$:

1. Calcular simbólicamente: $f'(x) = \frac{d}{dx} f(x)$.
2. Lambdificar ambas expresiones.
3. Generar dos `PlotCommand` de tipo `LINE_2D`.

3) Integral Gráfica (`plotintegral`)Integral Gráfica (plotintegral)

`plotintegral(.expr", a, b)` visualiza la integral definida:

1. Graficar $f(x)$ en un dominio extendido.
2. Sombrear la región $\int_a^b f(x)\,dx$ con `FILL_BETWEEN`.
3. Calcular numéricamente el área mediante la regla del trapecio:

$$\int_a^b f(x)\,dx \approx \sum_{k=0}^{n-2} \frac{f(x_k)+f(x_{k+1})}{2} \cdot (x_{k+1}-x_k)$$

4) Recta Tangente (`tangentline`)Recta Tangente (tangentline)

`tangentline(.expr", x₀)` calcula y grafica la recta tangente a $f(x)$ en $x = x_0$:

$$y_{\tan}(x) = f'(x_0)\cdot(x-x_0)+f(x_0)$$

donde $f'(x_0)$ se obtiene mediante diferenciación simbólica y evaluación puntual con SymPy.

E. Graficación 3-D

1) Superficies (`surface`)Superficies (surface)

Una superficie $z = f(x,y)$ se renderiza mediante:

1. Crear rejillas `meshgrid`: $X,Y = \text{meshgrid}(\text{linspace}(x_0,x_1,80),\text{linspace}(y_0,y_1,80))$.
2. Evaluar $Z = f(X,Y)$ vectorizadamente.
3. Renderizar con `gl.GLSurfacePlotItem` usando shader `"shaded"`.

La coloración por altura utiliza interpolación lineal entre azul ($z_{\text{mín}}$) y melocotón ($z_{\text{máx}}$):

$$\text{color}(z) = (1-t)\cdot\text{azul} + t\cdot\text{melocotón}, \quad t = \frac{z - z_{\text{mín}}}{z_{\text{máx}} - z_{\text{mín}}}$$

Específicamente: $R = 0{,}537 + 0{,}443t$, $G = 0{,}706 - 0{,}161t$, $B = 0{,}980 - 0{,}451t$, $\alpha = 0{,}85$.

2) Wireframes (`wireframe`)Wireframes (wireframe)

El wireframe utiliza el mismo `GLSurfacePlotItem` pero con `drawFaces=False` y `drawEdges=True`, mostrando únicamente las aristas de la malla.

3) Curvas Paramétricas 3-D (`parametric3d`)Curvas Paramétricas 3-D (parametric3d)

Una curva paramétrica $\gamma(t) = (x(t), y(t), z(t))$ se renderiza mediante `gl.GLLinePlotItem`:

$$\text{pts}_k = [x(t_k), y(t_k), z(t_k)]^T, \quad k = 0,\ldots,n-1$$

4) Superficies Paramétricas (`surfparam`)Superficies Paramétricas (surfparam)

Una superficie paramétrica $\sigma(u,v) = (x(u,v), y(u,v), z(u,v))$ se discretiza en una rejilla 60×60 y se renderiza mediante `gl.GLMeshItem`:

1. Generar vértices: $V = [X.\text{ravel}(), Y.\text{ravel}(), Z.\text{ravel}()]^T$.
2. Generar caras triangulares: para cada celda (i, j), dos triángulos:
   - $[i \cdot n + j,\ i \cdot n + j + 1,\ (i+1) \cdot n + j]$
   - $[i \cdot n + j + 1,\ (i+1) \cdot n + j + 1,\ (i+1) \cdot n + j]$
3. Colorear cada cara según la altura promedio de sus vértices.

5) Barras 3-D (`bar3d`)Barras 3-D (bar3d)

Cada barra se modela como un paralelepípedo (caja) con 8 vértices y 12 caras triangulares, renderizado con `gl.GLMeshItem`.

F. Ciclado Automático de Colores

Ambos renderizadores implementan un ciclo de 10 colores inspirado en la paleta **Catppuccin Mocha**:

$$\text{color}_i = \text{CYCLE}[i \text{ mód } 10]$$

Los colores fueron seleccionados para maximizar la distinguibilidad perceptual en fondos oscuros.

V. INTERFAZ GRÁFICA DE USUARIO

A. Arquitectura de la Ventana Principal

`MainWindow` implementa un diseño basado en **QDockWidget** con paneles acoplables:

- **Panel Editor** (izquierda superior): Editor con resaltado de sintaxis, números de línea, historial de comandos y autocompletado.
- **Panel de Salida** (izquierda inferior): Consola de resultados con diferenciación visual de errores.
- **Canvas** (centro): Widget de graficación dual 2-D/3-D con conmutación automática.

B. Resaltado de Sintaxis

`GraphUGHighlighter` aplica coloración basada en expresiones regulares con la paleta Catppuccin:

Token	Color	Hex
Comentarios `#...`	Gris tenue	`#6c7086`

Token	Color	Hex
Cadenas `"..."`	Verde	`#a6e3a1`
Números	Melocotón	`#fab387`
Palabras clave lógicas	Malva	`#cba6f7`
Constantes	Amarillo	`#f9e2af`
Funciones integradas	Azul	`#89b4fa`
Asignación `=`	Rosa	`#f5c2e7`
Operadores	Cian	`#89dceb`
Corchetes	Crema	`#f5e0dc`

C. Sistema de Temas

GraphUG incluye **5 temas** predefinidos: Catppuccin Mocha, Catppuccin Latte, Nord, Dracula y Solarized Dark. Cada tema se define como un `Palette` dataclass con 22 propiedades de color. El `ThemeManager` genera QSS completo a partir de la paleta y notifica a los componentes no-QSS (canvas PyQtGraph, resaltador de sintaxis).

D. Diálogos de Inserción

Siete diálogos especializados facilitan la construcción de comandos:

1. **InsertVectorDialog**: Inserción guiada de vectores.
2. **InsertMatrixDialog**: Constructor visual de matrices con dimensiones configurables.
3. **InsertPlotDialog**: Selector de primitivas de graficación con formularios contextuales.
4. **InsertCalculusDialog**: Selector de operaciones de cálculo y álgebra.
5. **InsertSnippetDialog**: Biblioteca de fragmentos de código predefinidos.
6. **SettingsDialog**: Configuración de tema y tamaño de fuente.
7. **AboutDialog**: Información del proyecto.

E. Sistema de Logging

El sistema de logging utiliza la biblioteca estándar `logging` de Python con:

- **Handler de consola** (`StreamHandler`): Nivel INFO, salida a `stderr`.
- **Handler de archivo rotativo** (`RotatingFileHandler`): Nivel DEBUG, archivos de 2 MiB con 3 respaldos, ubicados en `~/.graphug/logs/graphug.log`.

VI. DISCUSIÓN

A. Decisiones de Diseño

La elección de LALR(1) sobre PEG o parsers recursivos descendentes se justifica por:

1. **Rendimiento lineal garantizado** $O(n)$ — crítico para evaluación interactiva.
2. **Detección temprana de ambigüedades** en la gramática.
3. **Integración con Lark** que permite transformación inline durante el parsing.

La arquitectura MVC con inyección de dependencias explícita, aunque más verbosa que los enfoques monolíticos, proporciona:

- **Testabilidad**: Cada componente puede probarse de forma aislada.
- **Extensibilidad**: Nuevas primitivas de graficación o backends de evaluación pueden añadirse sin modificar el controlador.
- **Mantenibilidad**: La separación de capas reduce el acoplamiento.

B. Rendimiento y Escalabilidad

La vectorización NumPy permite evaluar funciones sobre arreglos de miles de elementos en microsegundos. La aceleración GPU mediante CuPy ofrece speedups significativos para operaciones de álgebra lineal en matrices grandes ($n > 5000$).

C. Limitaciones

- El parser no soporta definición de funciones por el usuario (`def`).
- No hay soporte para bucles (`for`, `while`).
- La graficación polar no muestra una rejilla polar nativa.
- Las superficies paramétricas 3-D tienen un costo computacional $O(n^2)$ en la generación de caras triangulares.

VII. CONCLUSIONES

GraphUG demuestra que es factible construir un entorno matemático interactivo completo utilizando Python y bibliotecas de código abierto. Las contribuciones principales son:

1. Un **lenguaje matemático con gramática LALR(1)** que soporta aritmética, álgebra lineal, cálculo simbólico y graficación en una sintaxis unificada.
2. Una **arquitectura MVC limpia** con interfaces abstractas e inyección de dependencias que facilita la extensión y el mantenimiento.
3. Un **sistema de renderizado dual** que conmuta automáticamente entre 2-D y 3-D según el tipo de primitiva.
4. Una **cobertura de pruebas exhaustiva** con 126+ tests unitarios organizados en fases incrementales.
5. Un **sistema de temas** configurable con 5 paletas predefinidas y generación automática de QSS.

El proyecto valida la viabilidad de combinar análisis sintáctico formal, computación numérica vectorizada, álgebra simbólica y graficación de alto rendimiento en una aplicación de escritorio moderna y extensible.

VIII. REFERENCIAS COMPLETAS

[1] A. V. Aho, M. S. Lam, R. Sethi, y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.ª ed. Boston, MA, EE. UU.: Pearson/Addison-Wesley, 2007.

[2] E. Shinan, "Lark — a parsing toolkit for Python," documentación oficial, 2024. [En línea]. Disponible: https://lark-parser.readthedocs.io/en/latest/

[3] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, sep. 2020. DOI: 10.1038/s41586-020-2649-2.

[4] A. Meurer *et al.*, "SymPy: symbolic computing in Python," *PeerJ Computer Science*, vol. 3, e103, ene. 2017. DOI: 10.7717/peerj-cs.103.

[5] The Qt Company, "Qt for Python (PySide6) Documentation," 2024. [En línea]. Disponible: https://doc.qt.io/qtforpython-6/

[6] L. Campagnola *et al.*, "PyQtGraph — Scientific Graphics and GUI Library for Python," documentación oficial. [En línea]. Disponible: https://pyqtgraph.readthedocs.io/en/latest/

[7] R. Okuta, Y. Unno, D. Nishino, S. Hido, y C. Loomis, "CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations," en *Proc. Workshop on ML Systems, NeurIPS 2017*, Long Beach, CA, 2017.

[8] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.

[9] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2017.

[10] Python Software Foundation, "Python 3.12 Documentation," 2024. [En línea]. Disponible: https://docs.python.org/3/

[11] IEEE, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019*, jul. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[12] G. Strang, *Introduction to Linear Algebra*, 5.ª ed. Wellesley, MA, EE. UU.: Wellesley-Cambridge Press, 2016.

[13] J. Stewart, *Calculus: Early Transcendentals*, 8.ª ed. Boston, MA, EE. UU.: Cengage Learning, 2015.

[14] E. W. Weisstein, "Isocurve," *MathWorld — A Wolfram Web Resource*. [En línea]. Disponible: https://mathworld.wolfram.com/Isocurve.html

[15] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3.ª ed. Reading, MA, EE. UU.: Addison-Wesley, 1997.

[16] Catppuccin Contributors, "Catppuccin — Soothing pastel theme," 2024. [En línea]. Disponible: https://catppuccin.com/

[17] M. L. Hetland, *Python Algorithms: Mastering Basic Algorithms in the Python Language*, 2.ª ed. Nueva York, NY, EE. UU.: Apress, 2014.

[18] Python Software Foundation, "PEP 634 — Structural Pattern Matching: Specification," 2021. [En línea]. Disponible: https://peps.python.org/pep-0634/

[19] The Khronos Group, "OpenGL 4.6 Core Profile Specification," 2023. [En línea]. Disponible: https://www.khronos.org/opengl/

[20] W. H. Press, S. A. Teukolsky, W. T. Vetterling, y B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3.ª ed. Cambridge, Reino Unido: Cambridge University Press, 2007.